

Compiler Construction Lab

Mini Compiler for a C-like Language

Project Report

Section: A



Submitted By:

Syed Shayan Arshad	2023-CS-625
Adeel Ul Rehman	2023-CS-626
Hammad Khalid	2023-CS-637
Waqar Haider	2023-CS-640

Submitted To:
Miss Ghazala

Dated: 7th June 2026

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

1. Abstract

This report presents the design and implementation of a mini compiler for a subset of the C language. The compiler implements six phases — lexical analysis, syntax analysis, semantic analysis, optimization, intermediate code generation, and execution — supported by a symbol table and an error handling and recovery mechanism. The language supports data types (int, float, bool), control structures (if-else, while, for), functions, arrays, and input/output operations. Built using Flex, Bison, and C++, the compiler is accompanied by a web-based GUI and an AI-powered error-fixing feature using Google Gemini. It correctly tokenizes, parses, and validates C-like source programs and detects both syntax and semantic errors.

2. Introduction and Objectives

A compiler translates source code written in a high-level language into a lower-level representation through a series of phases, each handling a specific aspect of the translation. The objective of this project is to design and implement a mini compiler for a C-like language. The compiler is required to:

- Support basic data types (int, float, bool).
- Handle control structures (if-else, while, for).
- Support functions with parameters and return values.
- Implement input/output operations and arrays.
- Process source code through all compiler phases.
- Integrate a symbol table and an error handling mechanism.

We adopted an incremental development approach, building and testing each phase in turn before integrating them into a complete pipeline with a web-based interface.

3. Features

Beyond the core requirements, the compiler implements the following features:

- **Core language** — Three data types (int, float, bool) and three control structures (if-else, while, for).
- **Functions** — Functions with parameters and return values.
- **Arrays** — 1D arrays with declaration, initialization, assignment, and access.
- **Input/Output** — input() and output() operations.
- **Symbol table** — Scope-aware symbol table with nested scopes and variable shadowing.
- **TAC** — Three Address Code generation with a direct TAC interpreter.
- **Optimization** — Compile-time constant folding of constant expressions.
- **Error recovery** — Multiple errors reported per run with meaningful messages.
- **Web GUI** — Web-based interactive editor with five tabs (Output, TAC, AST, Symbol Table, Errors), Compile & Run, Clear/Reset, and a Ctrl+Enter shortcut.
- **AI error fixing** — A “Fix with AI” button that sends the error and code to Google Gemini (gemini-2.0-flash), which returns a corrected version based on the language grammar.

4. Tools and Technologies

Component	Technology
Lexical analysis	Flex
Parser generation	Bison
Implementation language	C++

Component	Technology
Build automation	Make
Web backend	Flask (Python)
AI error fixing	Google Gemini (gemini-2.0-flash)

5. Language Specification

5.1 Data Types

Type	Description	Example
int	Integer values	int x = 5;
float	Floating-point values	float y = 3.14;
bool	Boolean values (0/1)	bool flag = true;
string	String literals	output("Hello");

5.2 Control Structures

Structure	Syntax	Example
If-else	if (cond) stmt else stmt	if (a > b) output(a);
While loop	while (cond) stmt	while (i < 10) i = i + 1;
For loop	for (init; cond; step) stmt	for (i=0; i<5; i=i+1) output(i);

5.3 Functions, Arrays and I/O

```
int add(int x, int y) { return x + y; }
int arr[5];   int arr[3] = {10, 20, 30};   arr[0] = 100;   output(arr[1]);
input(a);     output(a + b);
```

6. Compiler Implementation

6.1 Lexical Analyzer (Scanner)

The lexical analyzer breaks the source program into a stream of tokens using Flex. Tokens include keywords, identifiers, literals, and operators, and are forwarded to the parser.

Pattern	Token
[a-zA-Z_][a-zA-Z0-9_]*	IDENTIFIER
[0-9]+	INT_LIT
[0-9]+\.[0-9]+	FLOAT_LIT
\("[^"]*" \)	STRING_LIT
"if" / "while" / "for"	IF / WHILE / FOR

6.2 Syntax Analyzer (Parser)

The parser, generated with Bison, validates the token stream against the defined grammar and constructs an Abstract Syntax Tree (AST). Parse validation accepts all supported constructs and rejects malformed input.

Key Grammar Rules:

```
program      → declaration_list function_list
```

```

declaration → type IDENTIFIER [ '=' expr ] ';' | type IDENTIFIER '[' expr ']' ';'
function    → type IDENTIFIER '(' params ')' block
statement   → assignment | if | while | for | block

```

AST Node Types:

NODE_PROGRAM, NODE_FUNC, NODE_DECL, NODE_ASSIGN, NODE_IF,
 NODE_WHILE, NODE_FOR, NODE_BINARY, NODE_ARRAY_DECL,
 NODE_ARRAY_ACCESS.

6.3 Semantic Analyzer

The semantic analyzer traverses the AST to enforce language rules that grammar alone cannot capture:

- **Type checking** — validates declarations, assignments, function arguments, binary operations, and return statements for type compatibility.
- **Scope resolution** — resolves each identifier to the nearest enclosing scope and supports nested scopes and variable shadowing.
- **Declaration validation** — reports use of undeclared identifiers and redeclaration within the same scope.

6.4 Symbol Table

The symbol table stores every identifier together with its attributes (name, type, scope_level, is_array, array_size) and manages scope information through four operations:

Operation	Purpose
enter_scope()	Open a new (nested) scope on block entry
exit_scope()	Discard the current scope on block exit
insert_symbol()	Add an identifier to the current scope
lookup_symbol()	Resolve an identifier across enclosing scopes

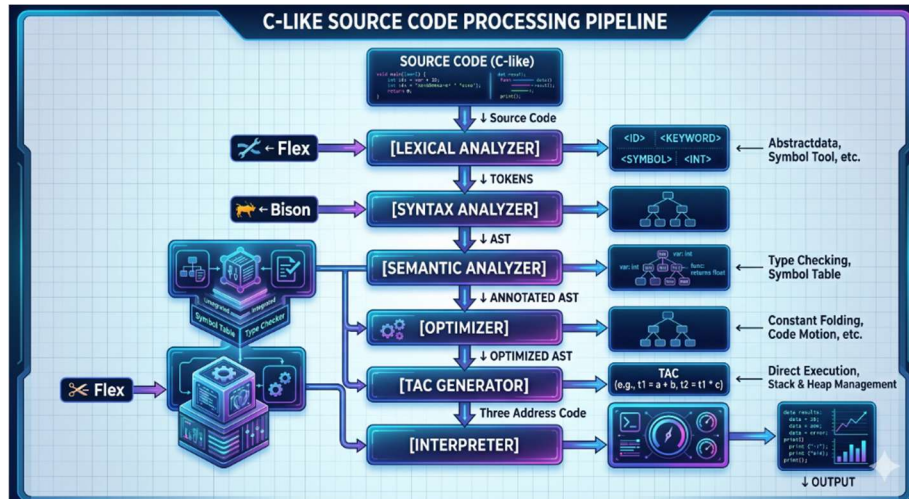
6.5 Intermediate Code and Optimization

The compiler generates Three Address Code (TAC) as an intermediate representation, where each instruction has at most one operator and three addresses (two operands and one result). Supported opcodes include TAC_ASSIGN, TAC_ADD, TAC_SUB, TAC_MUL, TAC_DIV, TAC_LABEL, TAC_GOTO, TAC_IFGOTO, TAC_ARRAY_STORE, TAC_ARRAY_LOAD, TAC_INPUT, and TAC_OUTPUT. A constant folding optimization evaluates constant expressions at compile time (for example, $x = 2 + 3 * 4$ becomes $x = 14$), and a TAC interpreter executes the generated instructions directly to produce program output.

6.6 Error Handling and Recovery

The compiler reports multiple errors in a single run rather than stopping at the first failure, and emits meaningful, descriptive messages. Three mechanisms work together:

- **Panic-mode recovery** — on a syntax error the parser skips tokens until a synchronizing token (; or }) is reached, then resumes.
- **Error productions** — dedicated grammar rules using the error token catch common malformed constructs.
- **Error collection** — an ErrorList structure accumulates all lexical, syntax, and semantic errors so they can be reported together.



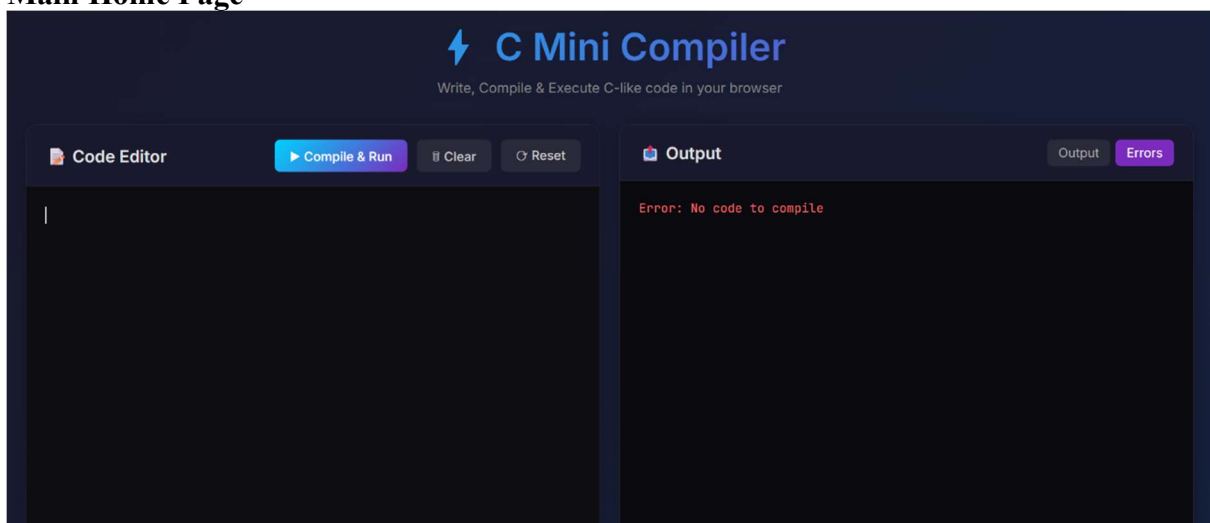
7. Test Cases and Results

The compiler was tested with programs that demonstrate both correct compilation and error detection. Each test produced the expected result.

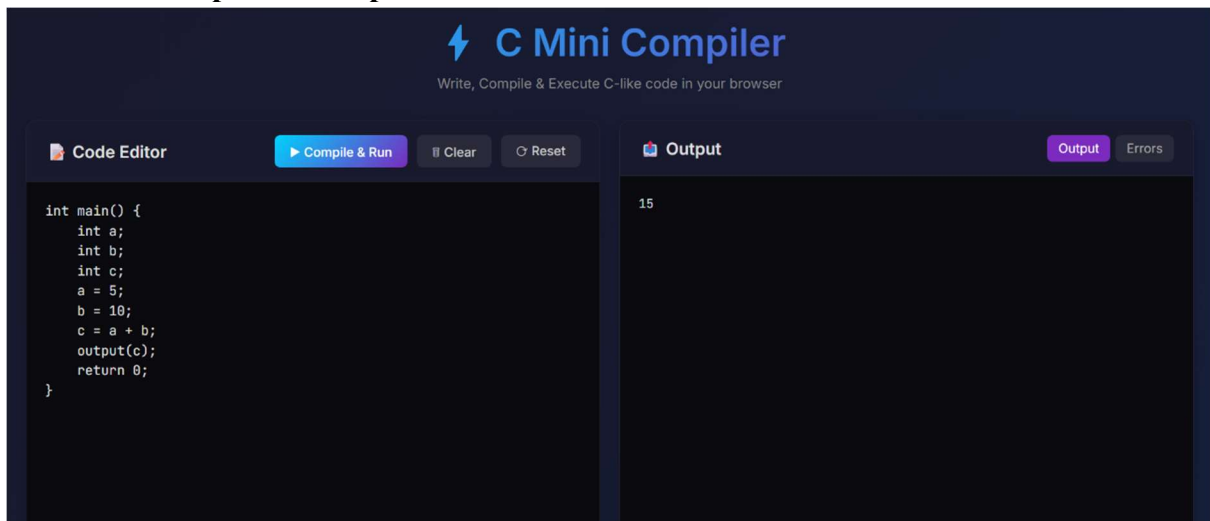
Test Case	Description	Expected	Actual
test_valid.c	Variable declaration & output	5	5
test_type_mismatch.c	float to int assignment	Semantic error	Type mismatch
test_scope.c	Variable shadowing	Parse success	Success
test_undeclared.c	Use of undeclared variable	Semantic error	Undeclared id
test_array_init.c	Array with initialization	10,20,30	10,20,30
test_array.c	Array loop assignment	0,10,20,30,40	0,10,20,30,40
test_constant_folding.c	2+3*4 optimization	14 in TAC	14 in TAC

8. GUI

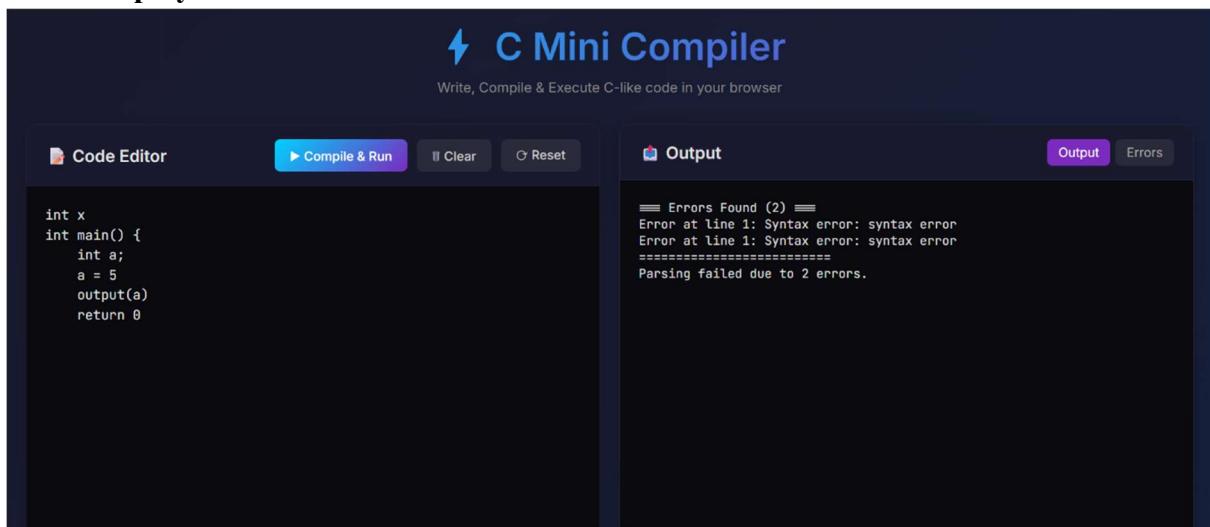
Main Home Page



Successful compilation output



Error display



9. Conclusion

The mini compiler was successfully designed and implemented, meeting all the required deliverables. It provides a working lexical analyzer, a syntax analyzer with a defined grammar and parse validation, a semantic analyzer performing type checking, scope resolution, and declaration validation, a scope-aware symbol table, and an error handling and recovery mechanism with meaningful messages. All six compiler phases were implemented and integrated. Additional features include intermediate code generation, constant folding optimization, a web-based GUI, and AI-powered error fixing. The compiler correctly compiles valid C-like programs and detects invalid ones, as confirmed by the test cases above.